

Packaged Model Instructions

To use the packaged model, first ensure you have Python 3.8+ with pandas, scikit-learn, and numpy installed. The trained model is saved as `final_model.pkl` using Python's pickle module. Input data must be a pandas DataFrame with exactly 483 feature columns corresponding to Windows API/system call counts (e.g., `NtCreateFile`, `RegOpenKey`), named as listed in `feature_name_to_number_mapping.csv`. All features must be numeric (missing/absent calls should be encoded as 0). Do not include target or `risk_score` columns in input data. The model outputs a binary prediction: 0 = low-risk executable (risk score < 30%), 1 = high-risk executable (risk score $\geq$ 30%). The test script (saved as `project2_test_script`) demonstrates this binary prediction by sampling 20 records from the dataset and printing both true and predicted risk classes side by side for validation.

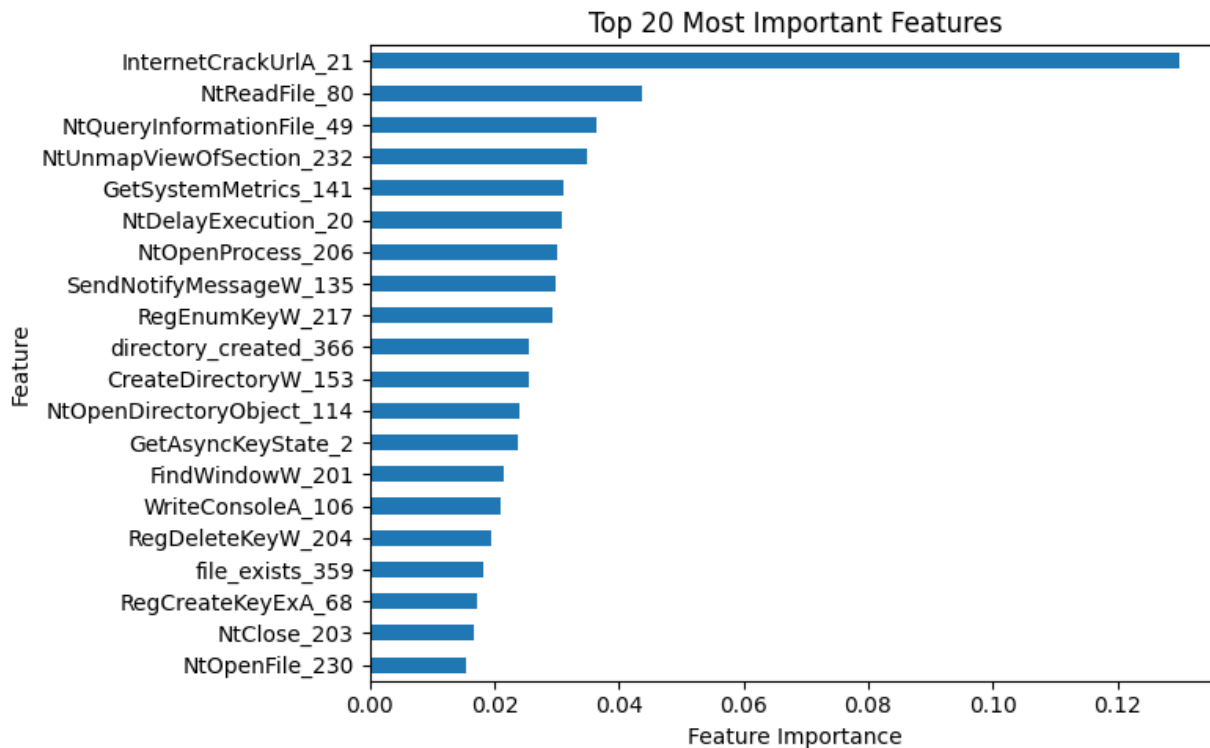
Data Preparation Summary

Raw data consisted of 45 monthly .txt files in sparse LIBSVM format (standard text-based file format), where each row represented an executable with a risk score (0–1) followed by active features in `feature_id:value` pairs. Missing features were absent from rows, implying a true value of zero. All files were parsed into a dense DataFrame, filled missing entries with 0, and enforced a consistent schema of 483 features (indexed `feature_0` to `feature_482`) based on domain knowledge. The binary target was derived by thresholding the risk score at 0.30. Finally, generic feature names were replaced with meaningful Windows API names using the provided `feature_name_to_number_mapping.csv`, resulting in an interpretable, modeling-ready dataset (`flat_file_labeled.csv`) with 483 numeric features and no missing values. Note that duplicate feature names were detected and handled by appending the corresponding feature number to enforce unique feature names. I didn't choose to delete these duplicate names because from the knowledge of telemetry data, the same API call can be instrumented in multiple contexts.

Model Summary

A Gradient Boosting Classifier (`sklearn.ensemble.GradientBoostingClassifier`) was trained with default hyperparameters and a fixed random state for reproducibility. The model leverages decision trees built sequentially to correct errors of prior trees, making it robust to feature interactions common in behavioral telemetry data. Performance was evaluated via 5-fold stratified cross-validation (preserving class balance) using precision and recall. These metrics are critical given the cost of false negatives (missing malware) and false positives (blocking benign software). The model achieved mean precision of 0.911 and recall of 0.974, indicating strong ability to identify high-risk executables while maintaining low false-positive rates.

The final model was retrained on the full dataset and saved for deployment. Feature importance analysis revealed that Part of WinINet API (`InternetCrackUrlA_21`), Low-level Windows NT API to read data from a file or device (`NtReadFile_80`), Metadata retriever (`NtQueryInformationFile_49`), strong indicator of process injection or hollowing (`NtUnmapViewOfSection_232`) and system configuration settings retriever (`GetSystemMetrics_141`) were among the top predictors respectively, aligning with known malware behaviors. The plot below shows the top 20 most important features:



Key Insights/Comments

- The data required extensive preprocessing due to its sparse, unstructured format. Converting from the raw LIBSVM-style format to a flat, labeled DataFrame ensured model compatibility, and the risk threshold of 0.3 effectively split executables into high and low-risk categories.
- Feature naming matters a lot. Duplicate API names in the mapping file caused critical pipeline errors. Enforcing unique, traceable feature names ensured consistency during training and testing.
- The Gradient Boosting Classifier provided interpretable feature importances that reflect real-world malware behaviors. Cross-validation also confirmed the model's stability and avoided overfitting.
- Deployment readiness was demonstrated by packaging the final model successfully into a pickle file (final_model.pkl) and validated via a standalone test script that loads new samples and produce predicted classifications alongside true labels.